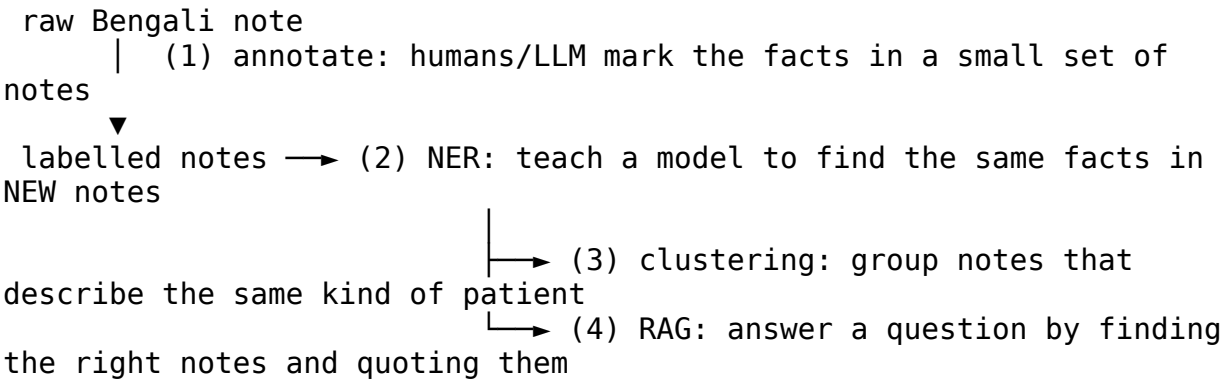
How our MedBhasha pipeline works — from the very basics

A companion guide for Team JUSL Health Home (read this before the finale)

Who this is for. Anyone on the team (or a judge) who wants to understand *what* the system does and *why*, without assuming prior machine-learning knowledge. Each section starts with the idea in plain words, then shows the real data and real numbers from our run, then points to the file where it lives.

1. The problem in one page

Doctors write notes in free text. A computer cannot "read" them like a person, so we want to **turn free text into structured facts** and then **use the facts**.



- **Why Bengali?** English clinical text has big open datasets (MIMIC, i2b2). Bengali — spoken by 200 million people — has almost none. The hackathon's real goal is to *create* such a dataset; NER, clustering and question answering are the proof that the dataset is useful.
- **What we hand in.** An annotated corpus (Round 1), and a notebook that does NER + clustering + question answering on 1 691 notes we had never seen (Round 2).

2. Ten words you need

Word	Plain meaning	Example from our data
Token	one "word" after splitting the text	মেটফরমিন, ৫০০, মি.গ্রা., ।
Entity / span	a group of tokens that names one fact	গত তিন দিন ধরে (a time), ডায়াবেটিস (a disease)
Entity type	the category of the fact; we have 10	DIAGNOSIS, SYMPTOM, MEDICATION, DOSAGE,

Word	Plain meaning	Example from our data
		TEST, TEST_RESULT, BODY_SITE, PROCEDURE, TEMPORAL, NEGATION_UNCERTAINTY
BIO tagging	how spans are written token by token: `B-` = Begins, `I-` = Inside, `O` = Outside	গত=B-TEMPORAL তিন=I-TEMPORAL দিন=I-TEMPORAL
NER	Named-Entity Recognition: predicting the BIO tags automatically	the model reads a new note and returns tags
Precision / Recall / F1	of what the model *found*, how much is right (P); of what *exists*, how much it found (R); F1 is their harmonic mean	found 80 spans, 70 right, 100 exist → P = 0.875, R = 0.70, F1 = 0.78
Entity-level F1	a span counts as correct only if **every** token and the type match	predicting দুই দিন when the truth is গত দুই দিন ধরে = wrong
Clustering	grouping items by similarity **without** being told the groups	notes of asthma patients end up together
NMI / ARI	scores comparing our groups with the hidden "true" groups (1 = perfect, ≈ 0 = random)	used by the organisers
RAG	Retrieval-Augmented Generation: first *retrieve* relevant notes, then *answer* using only them	question → 10 notes → answer with `[MB-0123]` citations

3. Step 1 — Data and annotation

What a training example looks like. One note, one row per token, one tag per token (`data/round1/annotated_corpus.csv`):

রোগীর	ডায়াবেটিস	আছে	।	সকালে	উপবাসে	রক্তে	শর্করা	১৮০	এসেছে	,	তিনি
O	B-DIAGNOSIS	O	O	B-TEMPORAL	O	B-TEST	I-TEST	B-TEST_RESULT	O	O	O

Written as markup (how we actually author notes, `data/round1/annotation_sources/`):

৫৫ বছর বয়সী পুরুষ রোগী। রোগী জানান, {TMP|গতকাল রাত থেকে} {BODY|বাম পায়ের বুড়ো আঙুলে} {SX|তীব্র ব্যথা}, {SX|ফোলা} ও {SX|লালচে ভাব}, {SX|জ্বর} {NEG|নেই}। পূর্ব ইতিহাস: মাংস ও অ্যালকোহল বেশি খাওয়ার অভ্যাস। {TEST|সিরাম ইউরিক অ্যাসিড} {RES|৯.৮ মি.গ্রা./ডেসিলিটার}। রোগ নির্ণয়: {DX|গাউট}। চিকিৎসা: {MED|ইন্ডোমেথাসিন} {DOS|২৫ মি.গ্র ...

Our corpus. 232 notes, 12060 tokens. They are *synthetic* (invented, no real patients) and were drafted by an AI assistant together with the team — allowed by the rules *if disclosed*. We disclose it, and we also disclose that **no clinician has checked the labels**.

Why 10 000 tokens? The organisers said that is roughly what a model needs. A model learns patterns from examples; with 612 tokens it had seen e.g. only 5 procedures. We measured the effect:

train_notes	train_tokens	precision	recall	F1
20	1065	0.843	0.775	0.808
40	2055	0.859	0.808	0.833
80	3973	0.891	0.847	0.868
120	6055	0.879	0.861	0.870
160	8306	0.914	0.894	0.904
192	10058	0.911	0.896	0.903

The validator. `resources/validate.py` is the organisers' checker: right columns, only legal tags, no `I-` without a `B-`. Ours prints *PASS, 0 errors, 0 warnings*.

4. Step 2 — Tokenizing (cutting text into tokens)

Rule (file `src/medbhasha/tokenizer.py`), the same in training and on the test notes:

1. split on spaces;
2. cut punctuation off the **ends** of a word (কাশি। → কাশি ।);
3. keep `-`, `/`, `.` **inside** words (১৪০/৯০, সর্দি-কাশি, ৮.২);
4. keep abbreviations like মি.গ্রা. whole.

Example: রোগীর গত দুই সপ্তাহ ধরে বুকে ব্যথা ও শ্বাসকষ্ট আছে। ইসিজি স্বাভাবিক, অ্যাসপিরিন ৭৫ মি.গ্রা. দিনে ১ বার দেওয়া হয়েছে। → রোগীর | গত | দুই | সপ্তাহ | ধরে | বুকে | ব্যথা | ও | শ্বাসকষ্ট | আছে | । | ইসিজি | স্বাভাবিক | , | অ্যাসপিরিন | ৭৫ | মি.গ্রা. | দিনে | ১ | বার | দেওয়া | হয়েছে | ।

5. Step 3 — NER with a CRF (teaching the computer to tag)

Idea. For every token the model decides among 21 tags (`O` plus `B-`/`I-` for each of 10 types). It looks at *clues* (called **features**) about the token and its neighbours, then picks the most likely **whole tag sequence** — that last part is what makes it a *Conditional Random Field*: it knows `I-DIAGNOSIS` cannot follow `O`, so it avoids impossible sequences.

Clues we give it for each token (file `src/medbhasha/ner.py`):

Clue	Why it helps	Value for the token প্যারাসিটামল
the word itself	জ্বর is almost always a symptom	প্যারাসিটামল
last 1–4 letters	Bengali adds endings (ডায়াবেটিসের); drug names share endings (-প্রাজল)	ামল
the shape	digits, `/`, `%`, Latin letters point to values and doses	`bb`
two words left and right	৫০০ followed by মি.গ্রা. is a dose	(neighbours)
gazetteer	our hand-written list of 762 drugs, diseases, tests … tells it প্যারাসিটামল is a drug even if never seen	`gaz:MEDICATION:B`
section	text after `Dx:` is probably a diagnosis, after `Rx:` a drug	`TREAT`

Trained on our corpus in about 10 seconds, deterministic. **Result on a new sentence** (the model has not memorised it):

র োগীর	গত	দুই	সপ্তাহ	ধরে	বুকে	ব্যথা	ও	শ্বাসকষ্ট	আছে	।	ইসিজি	স্বাভাবিক	,	অ্যাসপিরিন	৭৫	মি.গ্রা.	দিনে	১	বার	দেওয়া	হয়েছে	।
O	B-TEMPORAL	I-TEMPORAL	I-TEMPORAL	I-TEMPORAL	B-BODY_SITE	B-SYMPTOM	O	B-SYMPTOM	O	O	B-TEST	B-TEST_RESULT	O	B-MEDICATION	B-DOSAGE	I-DOSAGE	I-DOSAGE	I-DOSAGE	I-DOSAGE	O	O	O

How good is it? We hide part of our data, train on the rest, and score on the hidden part (cross-validation):

evaluation	model	precision	recall	F1
random 5-fold (by note)	full: lexicon + section features	0.888	0.864	0.876
random 5-fold (by note)	no gazetteer lexicon	0.866	0.828	0.847
random 5-fold (by note)	no section features	0.893	0.862	0.877
random 5-fold (by note)	neither (plain CRF)	0.873	0.825	0.848
domain-held-out 5-fold	full: lexicon + section features	0.873	0.849	0.861
domain-held-out 5-fold	no gazetteer lexicon	0.836	0.792	0.814
domain-held-out 5-fold	no section features	0.873	0.841	0.857
domain-held-out 5-fold	neither (plain CRF)	0.849	0.780	0.813

Domain-held-out means we hide whole specialties (all dermatology notes …) so the model must cope with diseases it never saw — closer to the hidden test. **Important:** because our notes are synthetic and written by one author, these scores are **too optimistic**; the real test will be lower.

What each part adds (the "no gazetteer" and "no section" rows above): the gazetteer helps most, because it carries medical knowledge that 12 000 tokens of training cannot.

Why not BERT / an LLM? (a) Too little data to train a big network; (b) pretrained Bengali models cannot be downloaded in a run that must work offline and reproduce *exactly*; (c) no external API is allowed. A CRF is small, fast, explainable and identical on every run.

6. Step 4 — Using the predictions on the 1 691 unseen notes

We run the model on `data/round2/test_notes.csv` and save the result three ways (`outputs/`): a token table (same layout as the training file), a span table with character positions, and JSONL. Summary:

37619 entity spans on 1691 of 1691 released notes (22.2 per note; our corpus has 12.5 per note).

entity type	spans	distinct forms	notes containing	share %
SYMPTOM	9097	535	1686	24.2
TEMPORAL	4962	132	1691	13.2
BODY_SITE	4930	246	1618	13.1
MEDICATION	4178	252	1656	11.1
TEST	3933	324	1351	10.5
DOSAGE	3404	212	1612	9.0
TEST_RESULT	2988	798	1111	7.9
DIAGNOSIS	2455	286	1486	6.5
NEGATION_UNCERTAINTY	1432	8	999	3.8
PROCEDURE	240	25	213	0.6

7. Step 5 — Clustering (finding groups without labels)

Idea. Notes about the same kind of patient mention similar entities. So: turn each note into a **list of numbers** that describes which entities it contains, then group notes whose lists are close.

1. **Keep only entities** the model found (never the note id, never the order in the file).
2. **Two views**: (A) the diagnosis text, as overlapping 2–4-letter chunks — so that আর্টিকারিয়া and আমবাত (আর্টিকারিয়া) still look similar; (B) a bag of diagnosis/symptom/drug/test/procedure/body-site phrases.
3. Weigh words by **TF-IDF** (rare, informative words count more than common ones), shrink to 64 numbers per note (**SVD**), and measure closeness by angle (cosine).
4. Try **k-means**, **Ward** (a bottom-up merge) and **Louvain** (community finding on a similarity graph) with many group counts.
5. **Choose without labels** using two self-check scores: *silhouette* (are points closer to their own group than to others? higher is better) and *Davies–Bouldin* (are groups compact and well separated? lower is better).

Result: ward, **90 clusters**, silhouette 0.777, Davies–Bouldin 0.717.

Two example clusters (diagnoses and drugs the model found in them):

cluster	size	top_diagnoses	top_medications
0	43	অ্যালার্জিক রাইনাইটিস (22); মৌসুমি অ্যালার্জিক রাইনাইটিস (10)	সিটিরিজিন সিরাপ (15); স্যালাইন নাসাল ড্রপ (15)
1	37	আর্টিকারিয়া (20); তীব্র আর্টিকারিয়া (13)	ক্যালামাইন লোশন (27); সিটিরিজিন সিরাপ (15)
2	36	তীব্র ওটাইটিস মিডিয়া (18); কান সংক্রমণ (9)	অ্যামোক্সিসিলিন সিরাপ (17); প্যারাসিটামল সিরাপ (17)

How the organisers score it. They compare our groups to hidden "phenotype" labels with **NMI** and **ARI**. We cannot compute those; as a *sanity check only* we compared with the diagnosis line printed inside each note: NMI = 0.905, ARI = 0.565 (on 1315 notes that print a diagnosis line) — this is **not** the real score.

Frequency and relationship analysis. From the predicted entities we also count how often each entity occurs and which diagnosis goes with which drug (*lift* > 1 means "more often together than chance"):

diagnosis	medication	co_notes	n_a	n_b	lift	pmi
ডায়াবেটিস	প্যান্টোপ্রাজল	24	137	76	3.890	1.960
উচ্চ রক্তচাপ	প্যারাসিটামল	24	115	358	0.980	-0.020
ডায়াবেটিস	প্যারাসিটামল	24	137	358	0.830	-0.280
জল শূন্যতা	ওআরএস	23	25	117	13.270	3.730
জল শূন্যতা	জিঙ্ক সিরাপ	22	25	22	67.520	6.080

diagnosis	medication	co_notes	n_a	n_b	lift	pmi
ডায়াবেটিস	ডাইক্লোফেনাক	22	137	72	3.760	1.910

8. Step 6 — Question answering with retrieval (RAG)

Idea. A question like *"HbA1c ৯-এর বেশি, মেটফর্মিন নিচ্ছেন এমন T2DM রোগী খুঁজুন"* is really a **filter**: diabetes **and** metformin **and** HbA1c > 9. We:

1. **Index** every note: normalised words, which medical *concepts* it mentions, patient age/sex, and its entities.
2. **Understand the query**: find its concepts (`diabetes`, `metformin`, `hba1c`), a numeric rule (`HbA1c > 9`), and who (child? elderly? male/female?).
3. **Match carefully.** A note that says *"ডায়াবেটিস নেই"* (no diabetes) or *"মায়ের ডায়াবেটিস"* (mother's diabetes) must **not** count. Spelling variants (মেটফর্মিন / মেটফরমিন) and synonyms (`PPI` = প্রোটন পাম্প ইনহিবিটর = ওমিপ্রাজল) must.
4. **Rank**: notes that satisfy **all** conditions first (concept coverage + word coverage + BM25 text score); if few do, we also show the nearest partial matches and say what is missing.
5. **Answer without inventing.** The answer is built from sentences and entities **copied** from the retrieved notes, each with its `[note-id]`. If nothing matches, it says so.

Real output for that query (shortened):

নথিতে ৮ জন রোগীর নোট পাওয়া গেছে যেখানে প্রশ্নের সব শর্ত মিলেছে।
রোগীভিত্তিক বিবরণ (উৎস নোট সহ):
- [n048] ৫০ বছর, পুরুষ — রোগ নির্ণয়: টাইপ ২ ডায়াবেটিস মেলিটাস — ওষুধ: মেটফর্মিন ৫০০ মি.গ্রা. দিনে ২ বার খাবারের পর, গ্লিমেপিরাইড ১ মি.গ্রা. সকালে ১টি — নোট থেকে: «খালি পেটে রক্তে শর্করা ১৮৫ মি.গ্রা./ডেসিলিটার, HbA1c ৯.২% | HbA1c ৩ মাস পর পুনরায় পরীক্ষা করুন»
- [n192] ৫২ বছর, পুরুষ — রোগ নির্ণয়: ডায়াবেটিস, অনিয়ন্ত্রিত টাইপ ২ ডায়াবেটিস — ওষুধ: মেটফর্মিন ১ গ্রাম দিনে ২ বার, ইনসুলিন রাতে ১০ ইউনিট — নোট থেকে: «HbA1c ১০.২%, খালি পেটে শর্করা ২১০ মি.গ্রা./ডেসিলিটার | HbA1c ৩ মাস পর»

Which notes may we search? The rules say "your own annotated corpus". The queries ask for patients, which are in the released notes. So by default we search *our notes + the released notes annotated by our own model*, and we also provide a strict "team corpus only" answer file. We will ask the organisers which one they mean.

Live query at the finale: open the notebook's last RAG cell, or run

```
python scripts/ask.py "আপনার প্রশ্ন এখানে"                  # default knowledge base
python scripts/ask.py --team-only "আপনার প্রশ্ন"            # strict variant
python scripts/serve_app.py                                  # tiny local web page (optional bonus)
```

9. How it is guaranteed to reproduce

- a single random seed (42) everywhere; the CRF itself has no randomness;
- all library versions are pinned in `requirements-lock.txt`;
- `outputs/run_manifest.json` stores an MD5 hash of every output file;
- `python scripts/check_reproducibility.py` runs everything again with a different Python hash seed and checks that every hash is identical. **Verified (2026-11-11):** the notebook run and a second, independent full run of `scripts/run_pipeline.py` under `PYTHONHASHSEED=12345` produced byte-identical files for all 27 output files in `outputs/` (`scripts/check_reproducibility.py`: "REPRODUCIBLE"). The check also caught and led us to fix one hash-order dependence (tie ordering in the association tables).

10. How to run it

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
jupyter nbconvert --to notebook --execute --inplace
MedBhasha_Pipeline.ipynb   # ≈ 6 min, or open it and "Run All"
# headless alternative:  python scripts/run_pipeline.py        (--fast
skips the ablation CV)
python tests/test_basics.py
# sanity tests
```

11. Which file does what

You want to …	Look at
see the training data	`data/round1/annotated_corpus.csv`, `data/round1/annotation_sources/*.txt`
read the annotation rules we used	`docs/ANNOTATION_CONVENTIONS.md`
change how words are cut	`src/medbhasha/tokenizer.py`
change the NER clues	`src/medbhasha/ner.py`, `src/medbhasha/lexicon.py`
see how groups are made	`src/medbhasha/cluster.py`
add a synonym / abbreviation for retrieval	`src/medbhasha/concepts.py`
see how answers are written	`src/medbhasha/rag.py` (`generate_answer`)
find the final answers	`outputs/rag_answers.md`
find the NER predictions	`outputs/ner_predictions_bio.csv`

12. Likely questions from the judges — and honest answers

Why is your NER score on the hidden set probably lower than your CV score? Our data are synthetic and one author wrote all of them; the model may have learned the author's habits. We say this in the report.

Did you train on the test notes? No. NER is trained only on our own corpus. The test notes are used as unlabeled *input*. We looked at a few of them to learn the *format* (section headers like `C/O`, `Rx`) and say so openly; no text was copied (largest overlap with any training note: 0.28 Jaccard).

How did you choose the number of clusters? By silhouette and Davies–Bouldin only. No labels.

Is the answer hallucinated? It cannot be: every sentence is copied from a retrieved note and cited; if nothing matches, it says "no matching note".

What happens with a concept that is not in your synonym table? It falls back to word matching (BM25) with spelling normalisation, so a new disease or drug name still works if it is written in a similar way in the notes; only abbreviations/class names need table entries. We tested this on questions we did not have before.

Why a CRF and not a transformer? Offline + exact reproducibility + 12 k tokens; the CRF with a medical gazetteer is strong in that regime. With public pretrained weights allowed, we would add IndicBERT/MuRIL as a second model.

What would you do with one more week? (1) A clinician reviews 50 notes and settles the ambiguities; (2) add notes in the registers where the model is weak; (3) second model and ensemble; (4) evaluate the answers with a clinical advisor.

13. Glossary

Annotation – marking facts in text. **BM25** – a classic text-search score that rewards rare matching words and avoids favouring long documents. **Concept** – a medical idea with several names (diabetes = ডায়াবেটিস = T2DM). **CRF** – model that predicts a whole tag sequence. **Cross-validation** – train on part of the data, test on the rest, repeat. **F1** – balance of precision and recall. **Feature** – one clue given to a model. **Gazetteer** – list of known terms. **Lift** – how many times more often two things co-occur than by chance. **Louvain** – an algorithm that finds communities in a graph. **NMI/ARI** – scores that compare two groupings. **Silhouette / Davies–Bouldin** – label-free quality scores for clusters. **Stemming** – cutting inflection so রোগীদের and রোগী match. **SVD** – a way to compress many columns into few while keeping most information. **TF-IDF** – word weighting that favours rare, informative words.